Okta configuration request — email template

Fill the `<PLACEHOLDERS>`, delete the italic notes, and send to the Okta administrator. This is written for the **org authorization server** (no custom authorization server is created or required) and a **confidential Web app** — the two things this deployment specifically needs.

Security note: don't send the client secret back in plain email — use a secrets channel (vault, one-time-secret link, etc.). The rest is fine over email.

To: Okta administrator **Cc:** , **Subject:** Okta OIDC app request — Claude gateway (`<FQDN>`): Web app + groups

Hi ,

We're deploying an internal application (the Claude apps gateway, with a Grafana admin dashboard and an installer **download portal**) that signs users in via Okta OIDC against our org authorization server. Could you set up the following OIDC application and send back the details at the bottom? Happy to hop on a call.

1. Application — OIDC Web app (confidential)

- **Sign-in method:** OIDC — **Web Application** (this is important: it must be the *Web* type so it has a **client secret**. A Single-Page/Native/public app has no secret and will not work — both the gateway and Grafana are server-side confidential clients and require the secret).
- **Grant types: Authorization Code AND Refresh Token** — please enable **both**. The Refresh Token grant is REQUIRED: without it Okta silently ignores the `offline_access` scope, issues no refresh token, and every developer is bounced back through the browser login at the gateway's session TTL (as short as 1 hour) instead of refreshing transparently. (Our clients also send PKCE (S256) — fine to leave PKCE allowed; it's used *in addition to* the secret, not instead of it.)
- **Scopes** / `offline_access`**:** the gateway requests `openid profile email offline_access groups`. `offline_access` **must be a granted scope on the app** (it is what mints the refresh token; it only works once the Refresh Token grant above is enabled). On the org authorization server, also confirm the refresh-token policy actually issues them.
- **Sign-in redirect URIs** — register **all three** on this one app:
- `https://<FQDN>/oauth/callback` (the gateway)
- `https://<FQDN>/grafana/login/generic_oauth` (the Grafana dashboard)
- `https://<FQDN>/portal/oauth/callback` (the installer download portal)
- **Sign-out redirect URI:** not required.

One app can cover the gateway, Grafana, and the portal (all three redirect URIs above). If you'd rather isolate them, a dedicated Web app per service also works — then we'd need a client ID/secret per app. The download portal defaults to its own dedicated Web app (clean secret rotation, least privilege) — if you set that up, register only `https://<FQDN>/portal/oauth/callback` *on it and send back a separate client ID/secret. Either is fine.*

2. Authorization server — the org server

- Use the **org authorization server** (issuer = our Okta domain, `https://<OKTA_DOMAIN>`). **Please do not create a custom authorization server** — our deployment is configured for the org server.

3. Groups — needed for dashboard role mapping and portal access

Both Grafana (role mapping) and the download portal (access gating) authorize on Okta group membership, so the app needs to return the user's groups:

- On the app's OIDC settings (on **each** app you create, if you split them), configure the **groups claim** so the user's Okta group memberships are returned when the `groups` scope is requested (the org server's built-in `groups` scope). A filter of **Matches regex** `.*` is simplest, or restrict to a prefix (e.g. groups starting with `grafana` / `claude`). Grafana and the portal both request the `groups` scope automatically. *(The portal checks the ID token first and falls back to the `/userinfo` endpoint, so either delivery works — but the groups claim must be configured on the app or the portal denies everyone.)*
- Create (or confirm) the admin group `<GRAFANA_ADMIN_GROUP>` (default name `grafana-admins`) and add our test user `<TEST_USER>` plus the intended dashboard admins. *(Role mapping is strict — a user in none of the mapped groups is denied the dashboard, so the test user must be in this group.)*
- Create (or confirm) the portal-access group(s) `<ACCESS_GROUP>` (default name `claude-gateway-users`; this may be one group or several) and add every developer who should be able to download the installer, including `<TEST_USER>`. *(The portal allows a member of any listed group and denies — and audits — everyone else.)*

4. Assignment & email domains

- **Assign** the app to the people who should sign in — the developer population for the gateway, and the admins above for the dashboard. Only assigned users can authenticate.
- The gateway restricts sign-in to specific email domains (`<ALLOWED_EMAIL_DOMAINS>`), so assigned users need an Okta profile email in one of those domains.

What to send back

1. **Client ID** (fine over email).
2. **Client Secret** — via a **secure** channel, not plain email.
3. Confirmation of the **issuer** (org server): `https://<OKTA_DOMAIN>`.
4. Confirmation that **groups are returned**: the easiest proof is the app / authorization-server **Token Preview** (or a real test login) for a user in `<GRAFANA_ADMIN_GROUP>`, showing a `groups` array in the token. *(Note: the groups won't appear in the `.well-known` discovery metadata — that's expected; it has to be checked from an actual token.)*
5. The exact **group name(s)** if they differ from `grafana-admins`.

Thanks very much — this is the last dependency before we can test end-to-end.

Best,

Placeholder cheat-sheet (delete before sending)

Placeholder	Value / where it comes from
`<FQDN>`	`GATEWAY_FQDN` in `scripts/deploy.env` (e.g. `claude-gateway.<domain>`)
`<OKTA_DOMAIN>`	your Okta org domain, e.g. `customerlogin.thecustomer.gov`
`<GRAFANA_ADMIN_GROUP>`	`GRAFANA_ADMIN_GROUP` in `deploy.env` (default `grafana-admins`)
`<ACCESS_GROUP>`	`ACCESS_GROUP` in `deploy.env` (default `claude-gateway-users`; gates the download portal)
`<TEST_USER>`	the user you'll log in with when validating the deployment
`<ALLOWED_EMAIL_DOMAINS>`	`ALLOWED_EMAIL_DOMAINS` in `deploy.env`

What you'll receive maps to `deploy.env` as: Client ID → `OKTA_CLIENT_ID` (and `GRAFANA_OKTA_CLIENT_ID` / `PORTAL_OKTA_CLIENT_ID` = the same if one app, or their own values if you registered dedicated apps); Client Secret → entered at the `set-okta-secret.sh` / `set-grafana-oidc-secret.sh` / `set-portal-oidc-secret.sh` prompts (never stored in `deploy.env`); issuer → `OKTA_ISSUER` (the bare domain, with `OKTA_AUTH_SERVER_TYPE=org`).